R ↔ Python

Command Correspondence Tables

Computational Biology — Data Analysis & Statistics

Python (numpy · pandas · matplotlib · seaborn · plotnine) vs R (base · ggplot2 · dplyr)

> **Key difference:** R vectors are 1-indexed (first element is [1]); Python arrays are 0-indexed (first element is [0]). This is the single most common source of off-by-one errors when translating code between the two languages.

1. General Commands

R	Python	Observations
`x <- 5` `x = 5`	`x = 5`	R prefers <- for assignment; = also works. Python uses = only.
`print(x)`	`print(x)`	Identical in both languages.
`cat('hello')`	`print('hello', end='')`	cat() in R prints without newline by default; use sep/fill args for control.
`# comment`	`# comment`	Single-line comments identical in both languages.
`?mean` `help(mean)`	`help(mean)` `mean.__doc__`	R's ? is shorthand for help(). Python uses help() or __doc__.
`rm(x)`	`del x`	Remove a variable from the environment/namespace.
`ls()`	`dir()`	List variables in current environment/namespace.

`getwd()`
`setwd('path')`
	`import os`
	`os.getcwd()`
	`os.chdir('path')`
		Working directory management.

`source('file.R')`	`exec(open('file.py').read())` `# or: import module`	Execute/load an external script or module.
`install.packages('pkg')` `library(pkg)`	`pip install pkg` `import pkg`	Package installation (terminal) and loading (script).
`class(x)`	`type(x)`	Get the type/class of an object.
`str(x)`	`print(x)` `x.info()  # for DataFrames`	Compact display of object structure. pandas .info() is closest for DataFrames.
`summary(x)`	`x.describe()`	Statistical summary of an object or DataFrame.
`length(x)`	`len(x)`	Number of elements. Note: for DataFrames, len() returns rows in pandas.
`nchar(x)`	`len(x)`	String length. R uses nchar(); Python uses len().
`paste(a, b)` `paste0(a, b)`	`a + b` `f'{a}{b}'`	String concatenation. paste() adds space by default; paste0() does not.
`sprintf('%d', x)`	`f'{x}'` `format(x)`	String formatting. Python f-strings are the modern equivalent.
`Sys.time()`	`import datetime` `datetime.datetime.now()`	Current date and time.

R	Python	Observations
`proc.time()`	`import time` `time.time()`	Elapsed time for benchmarking.

2. Mathematical Functions

Python: import math and/or import numpy as np required for most functions.

R	Python	Observations

`sqrt(x)`	`import math`
	`math.sqrt(x)`
	`# or: x**0.5`
		Square root. Python needs math module or ** operator.

`abs(x)`	`abs(x)`	Absolute value. Identical in both.
`exp(x)`	`import math` `math.exp(x)`	Exponential e^x.
`log(x)` `log(x, base)`	`math.log(x)` `math.log(x, base)`	Natural log. Both support a base argument.
`log10(x)` `log2(x)`	`math.log10(x)` `math.log2(x)`	Base-10 and base-2 logarithms.
`ceiling(x)` `floor(x)`	`math.ceil(x)` `math.floor(x)`	Ceiling and floor functions.
`round(x, n)`	`round(x, n)`	Round to n decimal places. Identical.
`x %% y`	`x % y`	Modulo (remainder). Different operator syntax.
`x %/% y`	`x // y`	Integer division. Different operator syntax.
`factorial(n)`	`math.factorial(n)`	Factorial.
`choose(n, k)`	`math.comb(n, k)`	Binomial coefficient (n choose k). math.comb available from Python 3.8+.
`max(x)` `min(x)`	`max(x)` `min(x)`	Maximum and minimum. Identical for scalars; use numpy for arrays.
`sum(x)`	`sum(x)` `np.sum(x)`	Sum of elements. Python built-in works on lists; numpy for arrays.
`cumsum(x)`	`import numpy as np` `np.cumsum(x)`	Cumulative sum. R is vectorized natively; Python needs numpy.
`prod(x)`	`import numpy as np` `np.prod(x)`	Product of all elements.
`sign(x)`	`np.sign(x)`	Sign of a number (-1, 0, 1).

`Inf`
`-Inf`
`NaN`
`NA`
	`float('inf')`
	`float('-inf')`
	`float('nan')`
	`None / np.nan`
		Special values. R's NA (missing) maps to np.nan in pandas contexts.

`pi`	`math.pi`	The constant π.

`sin(x)`
`cos(x)`
`tan(x)`
	`math.sin(x)`
	`math.cos(x)`
	`math.tan(x)`
		Trigonometric functions. Identical behaviour, different module.

3. Vector & Array Operations

Python equivalent: numpy arrays (import numpy as np). R vectors are natively vectorized.

R	Python (numpy)	Observations
`c(1, 2, 3)`	`import numpy as np` `np.array([1, 2, 3])`	Create a vector/array. R's c() is the fundamental building block.
`1:10`	`np.arange(1, 11)`	Integer sequence. R is inclusive; np.arange end is exclusive.
`seq(1, 10, by=2)`	`np.arange(1, 11, 2)`	Sequence with step. Conceptually identical.
`seq(0, 1, length.out=5)`	`np.linspace(0, 1, 5)`	Evenly spaced values. R: length.out; numpy: linspace.
`rep(x, n)` `rep(x, each=n)`	`np.repeat(x, n)` `np.tile(x, n)`	Repeat elements or whole vector.
`length(v)`	`len(v)` `v.shape[0]`	Number of elements.
`v[1]`	`v[0]`	Indexing. R is 1-based; Python is 0-based — a critical difference!
`v[2:5]`	`v[1:5]`	Slicing. R includes both ends; Python excludes the end index.
`v[c(1,3,5)]`	`v[[0,2,4]]`	Select multiple elements by index.
`v[v > 3]`	`v[v > 3]`	Boolean/logical indexing. Nearly identical with numpy arrays.
`v * 2` `v + 10`	`v * 2` `v + 10`	Vectorized arithmetic. Both languages apply operations element-wise.
`v1 + v2`	`v1 + v2`	Element-wise addition of two vectors/arrays.

`sum(v)`
`mean(v)`
`sd(v)`
	`np.sum(v)`
	`np.mean(v)`
	`np.std(v, ddof=1)`
		Aggregation. Note: numpy std uses ddof=0 by default (population); use ddof=1 for sample sd.

`var(v)`	`np.var(v, ddof=1)`	Variance. Same ddof note applies.
`median(v)`	`np.median(v)`	Median.
`quantile(v, 0.25)`	`np.quantile(v, 0.25)`	Quantiles/percentiles.
`sort(v)` `rev(v)`	`np.sort(v)` `v[::-1]`	Sorting and reversing.
`order(v)`	`np.argsort(v)`	Returns indices that would sort the vector.
`which(v > 3)`	`np.where(v > 3)[0]`	Indices of elements matching a condition.
`unique(v)`	`np.unique(v)`	Unique values.
`table(v)`	`import pandas as pd` `pd.Series(v).value_counts()`	Frequency table / value counts.
`append(v, x)`	`np.append(v, x)`	Append element(s). Note: numpy append returns a new array.
`is.na(v)`	`np.isnan(v)` `pd.isna(v)`	Detect missing values.
`na.omit(v)`	`v[~np.isnan(v)]`	Remove missing values.
`lapply(lst, fun)`	`[fun(x) for x in lst]` `list(map(fun, lst))`	Apply a function to each element; always returns a list. Python list

R	Python (numpy)	Observations
		comprehensions or map() are the natural equivalent.
`sapply(lst, fun)`	`np.array([fun(x) for x in lst])` `list(map(fun, lst))`	Like lapply but simplifies result to vector/matrix if possible. No direct equivalent; use a list comprehension + np.array().
`vapply(lst, fun, type)`	`np.array([fun(x) for x in lst])`	Like sapply but with a declared return type for safety. Python has no direct equivalent; numpy array creation enforces type implicitly.

`tapply(v, grp, fun)`	`pd.Series(v).groupby(grp).agg(fun)`
	`# or with DataFrame:`
	`df.groupby('grp')['v'].agg(fun)`
		Apply a function to subsets of a vector defined by a grouping factor. Closest Python equivalent is pandas groupby().agg().

4. Data Frame Operations

Python equivalent: pandas DataFrames (import pandas as pd).

R	Python (pandas)	Observations
`data.frame(a=c(1,2), b=c('x','y'))`	`import pandas as pd` `pd.DataFrame({'a':[1,2],'b':['x','y']})`	Create a data frame. Conceptually identical.
`read.csv('file.csv')`	`pd.read_csv('file.csv')`	Read CSV file into a data frame.
`write.csv(df, 'file.csv')`	`df.to_csv('file.csv', index=False)`	Write data frame to CSV.

`nrow(df)`
`ncol(df)`
`dim(df)`
	`len(df)`
	`len(df.columns)`
	`df.shape`
		Dimensions of a data frame.

`colnames(df)` `names(df)`	`df.columns`	Column names.
`rownames(df)`	`df.index`	Row names / index.
`head(df)` `tail(df)`	`df.head()` `df.tail()`	First/last rows. Both default to 6 (R) or 5 (Python).
`df$column` `df[['column']]`	`df['column']` `df.column`	Access a column.
`df[1, ]`	`df.iloc[0]`	Access first row. R is 1-based; Python iloc is 0-based.
`df[, 1]`	`df.iloc[:, 0]`	Access first column by position.
`df[df$age > 30, ]`	`df[df['age'] > 30]`	Filter rows by condition.
`subset(df, age > 30, select=c(name,age))`	`df.loc[df['age']>30, ['name','age']]`	Filter rows and select columns simultaneously.
`df$new_col <- df$a * 2`	`df['new_col'] = df['a'] * 2`	Add or modify a column.
`merge(df1, df2, by='id')`	`pd.merge(df1, df2, on='id')`	Join two data frames. R: merge(); pandas: merge().

R	Python (pandas)	Observations
`rbind(df1, df2)`	`pd.concat([df1, df2])`	Stack data frames vertically (append rows).
`cbind(df1, df2)`	`pd.concat([df1,df2], axis=1)`	Join data frames side by side (append columns).
`tapply(df$y, df$grp, mean)`	`df.groupby('grp')['y'].mean()`	Group-wise aggregation. tapply is closest; dplyr group_by is more similar.
`aggregate(y~grp, df, mean)`	`df.groupby('grp').agg({'y':'mean'})`	Aggregate with formula syntax vs dictionary.

`library(dplyr)`
`df %>% group_by(g) %>%`
`summarise(m=mean(y))`
	`df.groupby('g').agg(m=('y','mean'))`	Modern grouped summaries. dplyr syntax is very similar to pandas.

`reshape2::melt(df)`	`df.melt()`	Wide to long format transformation.
`tidyr::pivot_wider(df,...)`	`df.pivot()`	Long to wide format transformation.
`is.na(df)`	`df.isna()`	Detect missing values across entire data frame.
`na.omit(df)`	`df.dropna()`	Drop rows with any missing values.
`df[is.na(df)] <- 0`	`df.fillna(0)`	Replace missing values.
`order(df$col)`	`df.sort_values('col')`	Sort data frame by column.
`str(df)` `summary(df)`	`df.info()` `df.describe()`	Structural overview and statistical summary.

5. Plotting

Python options: matplotlib (low-level), seaborn (high-level), or plotnine (ggplot2 port).

R (ggplot2)	Python	Observations

`library(ggplot2)`	`import matplotlib.pyplot as plt`
	`import seaborn as sns`
	`from plotnine import *`
		Load plotting libraries.

`ggplot(df, aes(x=a, y=b)) +`
`  geom_point()`
	`plt.scatter(df['a'], df['b'])`
	`# or:`
	`sns.scatterplot(x='a',y='b',data=df)`
		Scatter plot.

`geom_line()`	`plt.plot(x, y)` `sns.lineplot(...)`	Line plot.
`geom_histogram(binwidth=5)`	`plt.hist(x, bins=20)` `sns.histplot(x, binwidth=5)`	Histogram.
`geom_bar(stat='count')`	`plt.bar(x, height)` `sns.countplot(x='col', data=df)`	Bar chart.
`geom_boxplot()`	`sns.boxplot(x='grp', y='val',` `data=df)`	Box plot. seaborn is closest to ggplot2 syntax.
`geom_violin()`	`sns.violinplot(x='grp', y='val',` `data=df)`	Violin plot.

R (ggplot2)	Python	Observations
`geom_smooth(method='lm', se=TRUE)`	`sns.regplot(x='a', y='b', data=df)`	Regression line with confidence band.
`geom_density()`	`sns.kdeplot(x)` `sns.kdeplot(data=df, x='col')`	Density / KDE plot.
`aes(color=group)` `aes(fill=group)`	`hue='group'  # in seaborn`	Map color to a grouping variable.
`aes(size=val)` `aes(alpha=val)`	`size=df['val']  # in plt.scatter` `alpha=0.5`	Map size or transparency to data.
`facet_grid(~ group)` `facet_wrap(~ group)`	`g = sns.FacetGrid(df, col='group')` `g.map(sns.histplot, 'x')`	Split plots into panels by a grouping variable.

`labs(title='T', x='X', y='Y')`	`plt.title('T')`
	`plt.xlabel('X')`
	`plt.ylabel('Y')`
		Axis labels and title.

`theme_minimal()` `theme_bw()`	`sns.set_theme(style='whitegrid')` `plt.style.use('ggplot')`	Overall visual theme.
`scale_x_log10()`	`plt.xscale('log')`	Logarithmic axis scale.
`xlim(0, 100)` `ylim(0, 50)`	`plt.xlim(0, 100)` `plt.ylim(0, 50)`	Set axis limits.
`scale_color_manual(values=c(...))`	`plt.set_palette([...])` `sns.set_palette([...])`	Custom color palette.
`ggsave('plot.png', dpi=300)`	`plt.savefig('plot.png', dpi=300)`	Save plot to file.

`# plotnine is ggplot2 in Python:`
`from plotnine import *`
`ggplot(df, aes(x='a',y='b')) +`
`  geom_point()`
	`# plotnine mirrors ggplot2 almost`
	`# exactly, including aes(), geom_*,`
	`# facet_*, labs(), theme_*`
		plotnine (pip install plotnine) is a direct Python port of ggplot2 with nearly identical grammar and syntax.

6. Flow Control

Key differences: R uses {} blocks and 'next'; Python uses indentation and 'continue'.

R	Python	Observations

`if (x > 0) {`
`  print('pos')`
`}`
	`if x > 0:`
	`  print('pos')`
		Basic if. R uses () for condition and {} for block; Python uses : and indentation.

`if (x > 0) {`
`  print('pos')`
`} else {`
`  print('non-pos')`
`}`
	`if x > 0:`
	`  print('pos')`
	`else:`
	`  print('non-pos')`
		if / else. Structurally identical logic; syntax differs.

`if (x > 0) {`
`  print('pos')`
`} else if (x == 0) {`
`  print('zero')`
`} else {`
`  print('neg')`
`}`
	`if x > 0:`
	`  print('pos')`
	`elif x == 0:`
	`  print('zero')`
	`else:`
	`  print('neg')`
		R uses 'else if' (two words); Python uses 'elif' (one word).

R	Python	Observations
`ifelse(x > 0, 'pos', 'neg')`	`'pos' if x > 0 else 'neg'`	Vectorized / inline ternary conditional. R's ifelse() works on whole vectors; Python's ternary is scalar.

`for (i in 1:10) {`
`  print(i)`
`}`
	`for i in range(1, 11):`
	`  print(i)`
		For loop over a sequence. R is 1-based and inclusive; Python range end is exclusive.

`for (x in c('a','b','c')) {`
`  print(x)`
`}`
	`for x in ['a', 'b', 'c']:`
	`  print(x)`
		Iterating over a vector/list of values. Conceptually identical.

`while (x > 0) {`
`  x <- x - 1`
`}`
	`while x > 0:`
	`  x -= 1`
		While loop. Same logic; syntax differs.

`repeat {`
`  x <- x - 1`
`  if (x <= 0) break`
`}`
	`while True:`
	`  x -= 1`
	`  if x <= 0:`
	`  break`
		Infinite loop with manual break. R has 'repeat'; Python uses 'while True'.

`# No do/while in R — use repeat:`
`repeat {`
`  do_something()`
`  if (!cond) break`
`}`
	`# No do/while in Python — use while True:`
	`while True:`
	`  do_something()`
	`  if not cond:`
	`  break`
		Neither R nor Python has a native do/while. Both emulate it with an infinite loop + break at the bottom.

`break`	`break`	Exit the current loop immediately. Identical in both languages.
`next`	`continue`	Skip to the next loop iteration. R uses 'next'; Python uses 'continue'.

`switch(x,`
`  'a' = print('got a'),`
`  'b' = print('got b')`
`)`
	`# Python 3.10+:`
	`match x:`
	`  case 'a': print('got a')`
	`  case 'b': print('got b')`
		Switch / match statement. R's switch() is a function; Python's match/case was added in 3.10.

`switch(x,`
`  'a' = 1,`
`  'b' = 2,`
`  99  # default`
`)`
	`match x:`
	`  case 'a': result = 1`
	`  case 'b': result = 2`
	`  case _:  result = 99`
		Switch with a default case. R: last unnamed value is default; Python uses case _.

`tryCatch({`
`  log(-1)`
`}, warning = function(w) {`
`  print('warning!')`
`}, error = function(e) {`
`  print('error!')`
`})`
	`try:`
	`  import math`
	`  math.log(-1)`
	`except ValueError as e:`
	`  print('error!')`
	`except Exception as e:`
	`  print('other error!')`
		Exception / error handling. R uses tryCatch(); Python uses try/except.

`tryCatch({`
`  risky()`
`}, error = function(e) e,`
`finally = {`
`  cleanup()`
`})`
	`try:`
	`  risky()`
	`except Exception as e:`
	`  pass`
	`finally:`
	`  cleanup()`
		The 'finally' block runs regardless of success or failure. Both languages support it.

`try(log(-1))`	`try:`
	`  math.log(-1)`
	`except: pass`
		Simplified error suppression. R's try() is a shorthand for tryCatch with no handler.

R	Python	Observations

`f <- function(x, y=1)`
`{`
`  return(x + y)`
`}`
	`def f(x, y=1):`
	`  return x + y`
		Function definition with a default argument. R uses <- and {}; Python uses def and :.

`# R returns last expression`
`# automatically:`
`f <- function(x) {`
`  x * 2  # returned implicitly`
`}`
	`# Python always needs explicit return:`
	`def f(x):`
	`  return x * 2`
		Implicit vs explicit return. R returns the last evaluated expression; Python always requires return.

`sapply(1:5, function(x) x^2)`	`list(map(lambda x: x**2, range(1,6)))` `# or: [x**2 for x in range(1,6)]`	Anonymous functions / lambdas applied to a sequence. Python list comprehensions are often preferred.
`function(x) x^2  # anonymous`	`lambda x: x**2`	Anonymous (unnamed) function syntax.